

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical - IX

Roll No.: T047	Name: Ashwin Tripathi
Class: TYIT	Batch: 1
Date of Assignment: 13-08-2026	Date/Time of Submission: 13-08-2026

Aim

- a. **Develop CRUD operations using ASP.NET Core Web API and Entity Framework Core.**

Code :

StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentAPI.Data;
using StudentAPI.Models;
```

```
namespace StudentAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentsController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public StudentsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET ALL
        [HttpGet]
        public IActionResult GetStudents()
        {
            return Ok(_context.Students.ToList());
        }

        // GET BY ID
        [HttpGet("{id}")]
```

```
public IActionResult GetStudent(int id)
{
    var student = _context.Students.Find(id);

    if (student == null)
        return NotFound();

    return Ok(student);
}

// INSERT
[HttpPost]
public IActionResult AddStudent(Student student)
{
    _context.Students.Add(student);
    _context.SaveChanges();

    return Ok(student);
}

// UPDATE
[HttpPut("{id}")]
public IActionResult UpdateStudent(int id, Student student)
{
    var data = _context.Students.Find(id);

    if (data == null)
        return NotFound();

    data.Name = student.Name;
    data.Age = student.Age;
    data.Course = student.Course;

    _context.SaveChanges();

    return Ok(data);
}

// DELETE
[HttpDelete("{id}")]
public IActionResult DeleteStudent(int id)
{

```

```

        var student = _context.Students.Find(id);

        if (student == null)
            return NotFound();

        _context.Students.Remove(student);
        _context.SaveChanges();

        return Ok();
    }
}
}

```

Program.cs

```

builder.Services.AddDbContext<AppDbContext>(options =>
{
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")
    );
});

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

```

appSetting.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=StudentDB;Trusted_Connection=True;"
  }
}

```

Students.cs

namespace StudentAPI.Models

```

{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public int Age { get; set; }
        public string Course { get; set; }
    }
}

```

AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using StudentAPI.Models;

namespace StudentAPI.Data
{
    7 references
    public class AppDbContext : DbContext
    {
        0 references
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        6 references
        public DbSet<Student> Students { get; set; }
    }
}

```

Output:

```

[
  {
    "id": 3,
    "name": "Ashwin Tripathi",
    "age": 20,
    "course": "IT"
  }
]

```

200

Response body

```

{
  "id": 3,
  "name": "Ashwin Tripathi",
  "age": 20,
  "course": "IT"
}

```

Response headers

Max Rows: 1000

	Id	Name	Age	Course
▶	3	Ashwin Tripathi	20	IT
⚙	NULL	NULL	NULL	NULL

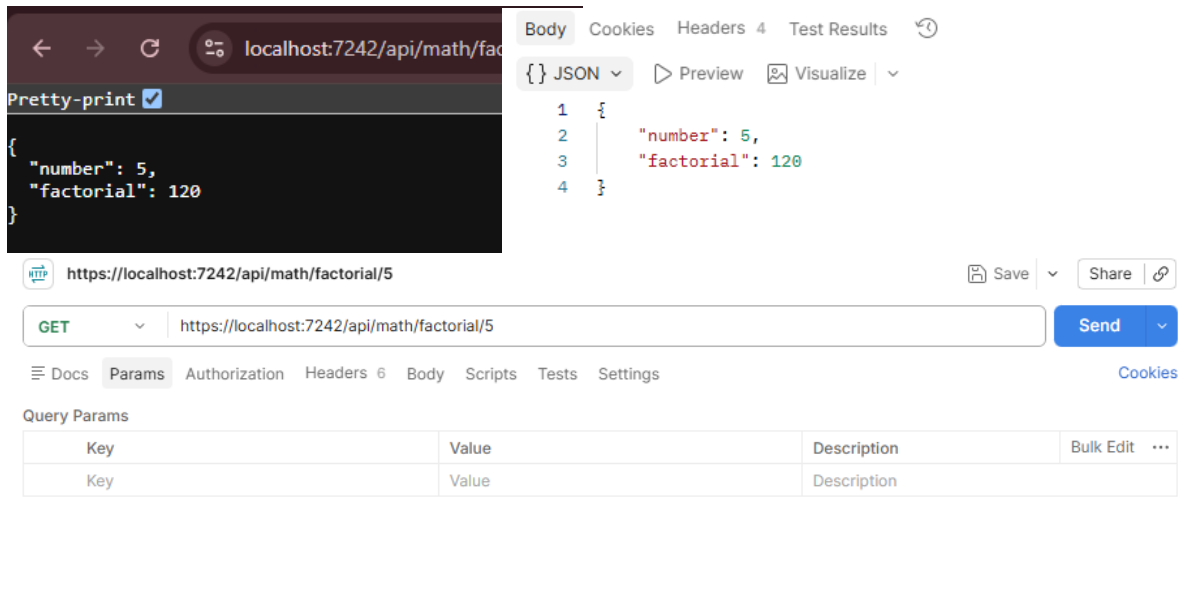
- b. Create an API endpoint to calculate Factorial of a Number and test it using Postman.

Code :**CODE:-**

```

using Microsoft.AspNetCore.Mvc;
namespace FactorialAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class MathController : ControllerBase
    {
        [HttpGet("factorial/{number}")]
        public IActionResult GetFactorial(int number)
        {
            if (number < 0)
            {
                return BadRequest("Factorial not possible for negative numbers.");
            }
            long factorial = 1;
            for (int i = 1; i <= number; i++)
            {
                factorial *= i;
            }
            return Ok(new
            {
                Number = number,
                Factorial = factorial
            });
        }
    }
}

```

OUTPUT:-


The screenshot shows a web browser at the URL `localhost:7242/api/math/factorial/5`. The response is a JSON object: `{\"number\": 5, \"factorial\": 120}`. The browser's developer tools are open, showing the JSON body of the response.

Below the browser window, there is a section for the API request details. The request is a GET request to `https://localhost:7242/api/math/factorial/5`. The response is a JSON object with the following structure:

```

{
  "number": 5,
  "factorial": 120
}

```

The response is displayed in the 'Body' tab of the developer tools. The 'JSON' tab is selected, showing the response in a pretty-printed format. The 'Preview' and 'Visualize' tabs are also visible.

At the bottom of the screenshot, there is a table for 'Query Params'.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		